PROTOFIRE

May 2026

# Smart contract audit

# SocietyProtocol

# Content

## Project description

The Society Protocol Contracts repository defines the Solidity contracts for Society Protocol, a modular on-chain badge, identity, community, and membership system. Unlike static NFT membership schemes or simple role registries, Society Protocol represents permissions, profiles, community roles, and reputation as configurable ERC1155 badges with programmable mint, transfer, burn, and balance logic. The system supports dynamic badge behavior through external hooks, staking-based personal VIP tiers, owner-granted community tiers, creator/member community badge flows, and non-transferable ERC20 wrapper tokens whose balances derive from badge ownership for integrations such as governance or Snapshot. The repository focuses on the core upgradeable contracts, factories, token primitives, and deployment scripts needed to create and manage badge-based communities and protocol-level reputation on-chain.

## Executive summary

| | |
|---|---|
| **Type** | Identity |
| **Languages** | Solidity |
| **Methods** | Architecture Review, Manual Review, Unit Testing, Functional Testing, Automated Review |
| **Documentation** | README.md |
| **Repositories** | https://github.com/SocietyProtocol/web3-app-contracts |

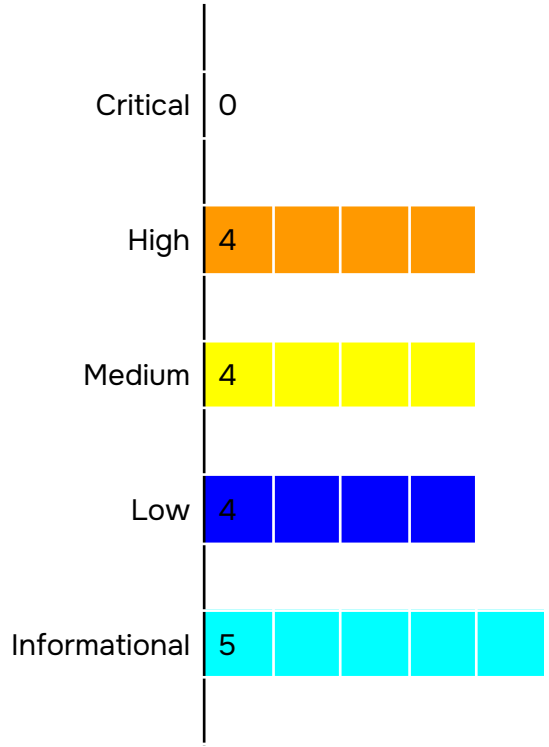
## Reviews

| Date | Repository | Commit |
|---|---|---|
| 01/05/26 | web3-app-contracts | c9b663b0f00a55bc41f41ecd7260ce86f9534134 |
| 13/05/26 | web3-app-contracts | b444df60b9835821b09896496da43273039f024e |

## Scope

| Contracts |
| --- |
| contracts/SPEC.sol<br>contracts/Accounts/SocietyVipManager.sol<br>contracts/Accounts/CommunityRegistry.sol<br>contracts/Accounts/CommunityWrapper.sol<br>contracts/Accounts/SocietyProtocolBadges.sol<br>contracts/Accounts/ISocietyBadgeHook.sol<br>contracts/Accounts/CommunityWrapperFactory.sol |

## Technical analysis and findings



# Security findings

## **** Critical

No critical severity issue found.

## *** High

### H01 - Expired VIP Lock Overwrites User Principal

SocietyVipManager allows users to lock ERC20 staking tokens in exchange for virtual VIP badge status. Each user has one lock record containing the locked amount and the unlock timestamp.

The issue occurs when a user's previous lock has already expired, but the user has not yet withdrawn the locked tokens with `unlock()`.

If the user calls `lock()` again while the previous expired lock is still recorded, the function treats the new lock as a fresh lock. It replaces the old recorded amount with the new amount instead of preserving or withdrawing the old principal.

A realistic flow:

1. Alice locks 100 tokens for 30 days.
2. After the lock expires, Alice does not call `unlock()`.
3. Alice calls `lock()` again with 50 tokens for another 30 days.
4. The contract records Alice's locked amount as 50 tokens.
5. The contract still physically holds Alice's original 100 tokens plus the new 50 tokens.
6. Alice's accounting record now only entitles her to withdraw 50 tokens.

The original 100 tokens are no longer associated with Alice's lock record.

**Impact:** This is a direct loss-of-funds accounting issue. Users can permanently lose access to previously locked principal by performing a natural action: relocking after expiry. The contract does not warn, revert, withdraw, or accumulate the expired amount. It silently overwrites the accounting state.

The stranded tokens remain in the VIP manager contract. They are not assigned to another user, but the original user no longer has a withdrawal path for them.

**Path:** contracts/Accounts/SocietyVipManager.sol:179

**Recommendation:** One of the solutions is to require users to withdraw expired locks before creating a new one.

If a user has a nonzero recorded amount and the lock has expired, `lock()` should revert with a clear error, such as `ExpiredLockMustBeUnlockedFirst`.

**Found in:** c9b663b0

**Status:** Fixed in b444df60

## H02 - createProfile() Reentrancy Can Mint Extra Profile Copies

SocietyProtocolBadges lets each user create a unique profile badge. The intended behavior is that each address can have one profile badge, and that profile badge should have a total supply of one.

During profile creation, the contract creates a new badge type, temporarily enables self-minting for that badge, records the new profile badge ID for the caller, mints one badge to the caller, and then removes the temporary self-mint permission.

The reentrancy issue happens during the mint step.

If the caller is a smart contract, ERC1155 minting invokes the recipient's `onERC1155Received()` callback. At that moment, the temporary self-mint permission is still active.

The contract has already set `profileBadgeId[msg.sender]`, so the recipient cannot call createProfile() again with a different profile ID. However, it can call the public `mint()` function for the same profile badge ID while the temporary self-mint permission still exists.

Because the profile badge temporarily allows `PERM_SELF`, and the recipient is minting to itself, the reentrant mint passes the permission check.

As a result, the contract recipient can mint additional copies of its profile badge before the original `createProfile()` call finishes, thereby removing the temporary permission.

Example flow:

1. A malicious or custom smart contract calls `createProfile()`.
2. The badge contract creates a new profile badge ID.
3. The badge contract temporarily allows self-minting for that profile badge.
4. The badge contract mints one profile badge to the caller.
5. During the ERC1155 receiver callback, the caller reenters `mint()` for the same profile badge ID.
6. The reentrant call succeeds because temporary self-minting is still enabled.

7. The profile badge ends with a supply greater than one.

**Impact:** This breaks the uniqueness invariant of profile badges.

Profile badges appear intended to be one-of-one identity tokens. Several parts of the contract assume this property. For example, `updateProfileURI()` uses total supply and holder balance checks to decide whether a caller is the profile owner.

If a profile badge has multiple copies, the semantics become unclear:

1. A profile may no longer represent a unique identity token.
2. Multiple copies could be distributed or retained by the same contract.
3. Profile metadata update assumptions can break.
4. Any future logic that assumes profile badge supply is exactly one may become unsafe.

This issue is limited to contract recipients because EOAs do not execute ERC1155 receiver callbacks. But contract wallets, account abstraction wallets, multisigs, and protocol-controlled accounts can all be contract recipients, so this is still a real execution path.

**Path:** contracts/Accounts/SocietyProtocolBadges.sol:268

**Recommendation:**
A good fix would be:

1. Add `mapping(uint256 => bool) isProfileBadge;`
2. In `createProfile()`, set `isProfileBadge[pid] = true` before minting.
3. In the `mint`/`update` path, if `isProfileBadge[id] and totalSupply(id) + amount > 1`, revert.

That directly blocks the reentrant extra mint even if temporary `PERM_SELF` is still active.

**Found in:** c9b663b0

**Status:** Fixed in b444df60

## H03 - Non-Profile Single-Supply Badges Can Have Metadata Rewritten

SocietyProtocolBadges has a function `updateProfileURI()` that is intended to let users update the metadata URI of their own profile badge.

The intended model appears to be:

- A user creates a profile.

- The profile is represented by a unique ERC1155 badge.
- The profile owner can later update that profile badge's metadata.

The issue is that `updateProfileURI()` does not verify that the badge ID passed to the function is actually the caller's profile badge.

Instead, it checks only two things:

1. The badge has a total supply of exactly one.
2. The caller owns one unit of that badge.

Those checks prove that the caller is the sole holder of a one-of-one badge. They do not prove that the badge is a profile badge. As a result, any badge with a total supply equal to one can be treated like a profile badge for metadata update purposes.

Example flow:

1. A one-of-one non-profile badge exists.
2. Alice is the sole holder of that badge.
3. Alice calls updateProfileURI() with that badge ID.
4. The contract checks that the total supply is one, and Alice owns one.
5. The contract updates the badge metadata URI.
6. The badge metadata changes even though Alice may not be an authorized editor for that badge.

This bypasses the normal editor model. Other metadata update functions, such as `setURI()` and `modifyBadge()`, rely on canEdit. But `updateProfileURI()` creates a separate metadata update path based only on supply and ownership.

**Impact:** This allows unauthorized metadata modification for non-profile one-of-one badges. A sole holder of a one-of-one badge could rewrite its URI to misleading, malicious, or unrelated metadata.

**Path:** contracts/Accounts/SocietyProtocolBadges.sol:505

**Recommendation:** `updateProfileURI()` should verify profile identity directly. The function should require that the badge ID equals the caller's registered profile badge ID.

**Found in:** c9b663b0

**Status:** Fixed in b444df60

## H04 - Malicious Hook on Pre-Referenced Future Badge IDs Grants Arbitrary Permissions

Badge creators can reference future (not-yet-created) badge IDs in their permission rules (`canMint`, `canTransfer`, `canBurn` arrays). The `createBadge()` function does not validate that referenced badge IDs actually exist. Badge IDs are predictable (sequential: `nextTokenId + 1, +2, ...`).

An attacker can exploit this by:
1. Observing an honest creator planning to create badge X with rule `canMint = [Y]` where `Y` is a future ID.
2. Front-running the creator and deploying badge `Y` themselves with a malicious hook.
3. Setting the malicious hook to return `balanceOf > 0` for any account.
4. When badge `X`'s permission check iterates through rules and calls `balanceOf(account, Y)`, the malicious hook grants permission to everyone.

Additionally, permission checks call `balanceOf()` (the overrideable version), which delegates to any installed hook. The hook is not constrained and can return arbitrary values.

**Impact:** - Anyone can mint, transfer, or burn a badge if its rules reference an attacker-controlled badge with a malicious hook.
- Attacker gains custodial control over every badge whose rules reference their badge ID.
- Permission system becomes untrusted.

**Path:** contracts/Accounts/SocietyProtocolBadges.sol

**Recommendation:** Consider:
- Validating that every ID in permission rules (`minters`, `transferers`, `burners`) exists (`_badgeCreated[id] == true`) and is not a future ID.
- Using the raw ERC1155 balance (`super.balanceOf()`) instead of the hook-aware `balanceOf()` for internal permission checks. This prevents hooks from altering trust decisions while keeping the public-facing `balanceOf()` hook-aware for read-only use cases.

**Found in:** c9b663b0

**Status:** Fixed in b444df60

## ** Medium

### M01 - Community Wrapper Ownership Is Orphaned

Community creators can deploy an ERC20-style `CommunityWrapper` for their community through `CommunityRegistry.deployCommunityWrapper()`.

The wrapper is meant to represent community membership based on ownership of certain ERC1155 badge IDs. It has owner-only functions such as adding and removing badge IDs from the wrapper's membership calculation.

The issue is that when a creator calls the registry to deploy a wrapper, the wrapper owner becomes the registry/factory call path, not the actual community creator.

The call flow is:

1. Community creator calls `CommunityRegistry.deployCommunityWrapper()`.
2. The registry calls `CommunityWrapperFactory.createWrapper()`.
3. Inside the factory, msg.sender is the registry contract, not the original creator.
4. The factory initializes the wrapper with `msg.sender` as `owner`.
5. Therefore, the wrapper owner becomes the registry.

The registry stores the wrapper address, but it does not expose functions that allow the community creator to manage the wrapper's badge set through the registry. As a result, the wrapper's owner-only management functions are effectively inaccessible to the community creator.

**Impact:** Community creators cannot manage their own community wrapper after deployment. This matters because `CommunityWrapper` includes owner-only functions to change which badge IDs count toward wrapper balance. If ownership is assigned to the registry, and the registry has no proxy methods for these actions, then the wrapper badge set is effectively frozen after deployment.

**Path:** contracts/Accounts/CommunityRegistry.sol:218, contracts/Accounts/CommunityWrapperFactory.sol:83

**Recommendation:** Pass the community creator as the wrapper owner during deployment. The registry already knows `msg.sender` when `deployCommunityWrapper()` is called, so it can pass that creator address into the factory, and the factory can initialize the wrapper with that address as owner.

**Found in:** c9b663b0

**Status:** Fixed in b444df60

## M02 - Community Creators Cannot Edit Badge Metadata or Settings

Community badges are created through the `CommunityRegistry`. This includes the creator badge, the member badge, and any additional community badges.
When the registry creates these badges, it sets the badge editor to the registry contract itself. The community creator is not assigned as an editor.

In `SocietyProtocolBadges`, badge metadata and configuration changes are controlled by the `canEdit` mapping. Functions such as `setURI()`, `setBadgeHook()`, and `modifyBadge()` require the caller to be an authorized editor for the badge.

Because the registry is the editor, direct calls from the community creator to the badge contract will fail. The creator does not have `canEdit` permission.

This could still work if the registry exposed creator-gated proxy functions, but it does not. The registry allows creators to update the community name and description stored in the registry, but it does not expose methods to update the ERC1155 badge URI, badge hook, badge name, official status, or other badge-level settings.

**Impact:** Community badge metadata and settings are effectively frozen after creation.

**Path:** contracts/Accounts/CommunityRegistry.sol:186, contracts/Accounts/CommunityRegistry.sol:198, contracts/Accounts/CommunityRegistry.sol:245

**Recommendation:** If community creators should directly manage badge metadata and settings, the registry should assign the creator as an editor when creating community badges. If the protocol wants edits to flow through the registry, then the registry should expose creator-gated proxy methods for badge administration. These methods should check `onlyCreator(communityId)` and then call the appropriate badge contract functions. Otherwise, consider documenting intended behavior.

**Found in:** c9b663b0

**Status:** Fixed in b444df60

## M03 - VIP Badge Safety Depends on Deployment Sequencing

VIP status is represented by badge IDs whose balances are controlled through the `SocietyVipManager` hook.

The VIP manager implements hook functions that deny direct minting, transferring, and burning of VIP badges. It also reports virtual balances based on how many staking tokens a user has locked.
This design only works if the VIP badge IDs in `SocietyProtocolBadges` are correctly configured to use the VIP manager as their hook.
The issue is that this relationship is not established atomically.
The VIP manager stores `bronzeBadgeId`, `silverBadgeId`, and `goldBadgeId`, but it does not verify that those badge IDs exist or that they are configured with the VIP manager hook.
The badge contract allows hooks to be set separately by badge editors. Until the correct hook is set, VIP badge behavior depends on the badge's ordinary permission rules. If those rules are permissive or incorrectly configured, VIP badges may be mintable, transferable, or burnable outside the staking system.

A safe deployment requires multiple steps to be performed correctly:

1. Create the VIP badge IDs.
2. Deploy and initialize the VIP manager with exactly those IDs.
3. Set each VIP badge's hook to the VIP manager.
4. Ensure the VIP badges do not have unsafe fallback permissions before the hook is active.

**Impact:** If deployment or configuration is misordered, VIP badge integrity can be broken.

Possible consequences include:

1. Users may mint VIP badges without locking tokens.
2. Users may transfer VIP badges even though VIP status is intended to be non-transferable.
3. VIP badges may be burned or manipulated outside the VIP manager.
4. VIP status shown by balanceOf() may not reflect staking state until the hook is correctly wired.

Even if the deployment is eventually corrected, any unauthorized minting or transfer that happened before hook wiring may create confusing historical state. For hooked balances, the hook may override visible balances, but underlying ERC1155 supply and transfer events could still be polluted.

**Path:** contracts/Accounts/SocietyVipManager.sol:130, contracts/Accounts/SocietyProtocolBadges.sol:349

**Recommendation:** Make VIP deployment wiring atomic or verifiable.

**Found in:** c9b663b0

**Status:** Fixed in b444df60. (The `initialize()` now validates that each VIP badge ID exists; `hook` wiring on those badges is handled by the deployment script).

## M04 - Invitation System Does Not Require Inviter To Be An Existing User

`SocietyProtocolBadges` includes an invitation system. A user can accept an invite by submitting an inviter's address, a signed message, and a signature. The function verifies that the signature was produced by the claimed inviter and that the message ends with the caller's address.
However, the contract does not verify that the inviter is actually an existing protocol user.

- There is no requirement that the inviter has a profile badge.
- There is no requirement that the inviter was previously invited.
- There is no requirement that the inviter hold any protocol credential.

As a result, any address can produce a valid invite signature for another user.

This weakens the intended purpose of the invitation graph. The comments describe the invite system as preventing bots by requiring a signature from a valid user, but the implementation only requires a signature from any address.

**Impact:** The invitation system does not establish a meaningful social graph unless off-chain systems impose additional rules. Anyone can create a fresh address and use it as an inviter. That means invite acceptance does not prove that the new user was invited by an existing participant.

**Path:** contracts/Accounts/SocietyProtocolBadges.sol:576

**Recommendation:** Define and enforce what qualifies someone to be an inviter.

**Found in:** c9b663b0

**Status:** Acknowledged

## * Low

### L01 - Tier Thresholds Can Be Changed Retroactively, Re-Tiering Existing Locks

The owner can call `setTierAmounts()` to change `bronzeAmount`, `silverAmount`, and `goldAmount` at any time. Existing user locks are not snapshots; their tier is computed dynamically by `onBalanceOf()` based on the current thresholds.

If the owner increases `goldAmount` from 1000 to 5000, a user who locked exactly 1000 tokens (previously Gold) would be instantly demoted to Silver.

**Impact:** - Users' tier status can unexpectedly change.
- Off-chain systems that cached tier status become stale.
- Governance power (if tiers are used for voting) can be silently reduced.

**Path:** contracts/Accounts/SocietyVipManager.sol:163-173

**Recommendation:** Consider adding a cooldown mechanism for the `setTierAmounts()` function.

**Found in:** c9b663b0

**Status:** Fixed in b444df60

### L02 - Lock Duration Has No Upper Bound

The `lock()` function only validates a minimum duration (`MIN_LOCK_DURATION = 30 days`) but has no maximum. A user could accidentally call `lock(amount, 100 years)` and have no way to correct it without waiting a century.

**Impact:** - User funds can be permanently locked if the `unlockTime` is set to a far-future timestamp.
- No way to unlock early or adjust duration.
- Accidental foot-guns can result in long-term capital loss.

**Path:** contracts/Accounts/SocietyVipManager.sol

**Recommendation:** Consider adding `MAX_LOCK_DURATION` validation.

**Found in:** c9b663b0

**Status:** Fixed in b444df60

## L03 - Per-Call Lock Minimum Unnecessarily Strict for Top-Ups

When a user has an active lock and wants to add more tokens, the `lock()` function requires the new `amount >= bronzeAmount`. If Bronze = 1000, Silver = 2000 and If a user locked `1900 SPEC`, they cannot add `100 SPEC` to reach `2000` because `100<1000`.

**Impact:** Users must always add at least Bronze amount, even for small top-ups.

**Path:** contracts/Accounts/SocietyVipManager.sol

**Recommendation:** Consider relaxing the check to enforce only the cumulative minimum: `if (userLock.amount + amount < bronzeAmount) revert InsufficientAmount()`.

**Found in:** c9b663b0

**Status:** Fixed in b444df60

## L04 - Unbounded Permission-Rule Arrays

`SocietyProtocolBadges` lets badge creators configure permission rules for minting, transferring, and burning. Each badge can have three rule arrays: `canMint`, `canTransfer`, `canBurn`.

These arrays are supplied during badge creation and are later iterated every time the badge is minted, transferred, or burned. There is no maximum length for these arrays.
This means a badge can be created with very large permission-rule arrays. Every future operation involving that badge may then require looping through many rules before determining whether the caller is authorized.

The permission loop can also call `balanceOf()` for badge-based rules. If those referenced badges use hooks, each permission check may trigger additional external hook logic.

**Impact:** A badge can become prohibitively expensive or unusable. If a badge has large rule arrays, minting, transferring, or burning that badge may consume too much gas and fail. This can effectively brick the badge's lifecycle operations.

**Path:** contracts/Accounts/SocietyProtocolBadges.sol:221, contracts/Accounts/SocietyProtocolBadges.sol:644

**Recommendation:** Set practical maximum lengths for permission-rule arrays.

**Found in:** c9b663b0

**Status:** Fixed in b444df60

## Informational

### I01 - Missing Storage Gaps in UUPS Upgradeable Contracts

Several contracts in the Accounts folder are UUPS upgradeable contracts. Upgradeable contracts preserve storage across implementation upgrades, so their storage layout must be managed carefully.

The affected contracts are:

- SocietyProtocolBadges
- SocietyVipManager
- CommunityRegistry
- CommunityWrapperFactory

None of these contracts declares a reserved storage gap.

A storage gap is an unused fixed-size array reserved at the end of an upgradeable contract's storage layout. It gives future implementations room to add state variables without accidentally colliding with storage introduced by parent contracts or future inheritance changes.

This is a common OpenZeppelin upgradeability practice. It is especially useful when contracts inherit from upgradeable base contracts that may evolve.

**Impact:** Future upgrades may become riskier or more constrained.

**Path:** contracts/Accounts/SocietyProtocolBadges.sol:18, contracts/Accounts/SocietyVipManager.sol:20, contracts/Accounts/CommunityRegistry.sol:15, contracts/Accounts/CommunityWrapperFactory.sol:15

**Recommendation:** Add explicit storage gaps to the UUPS upgradeable contracts.

**Found in:** c9b663b0

**Status:** Acknowledged

## I02 - Missing Strict Validation for Community Tier IDs

The `grantCommunityTier()` function's NatSpec explicitly documents that tierId should be one of three predefined values: `1 = Bronze, 2 = Silver, 3 = Gold`. However, the validation only checks that `tierId != 0`, allowing any positive number. The owner could set `tierId = 234` or any arbitrary value. Off-chain systems reading `getCommunityTier()` would not know what tier 234 represents.

**Path:** contracts/Accounts/SocietyVipManager.sol

**Recommendation:** Consider adding stricter validation that restricts tierId to documented values or updating natspec documentation with clear explanation.

**Found in:** c9b663b0

**Status:** Fixed in b444df60

## I03 - Badge Editors Cannot Be Rotated

`SocietyProtocolBadges` uses the `canEdit` mapping to decide who can modify badge metadata and settings. Editors are assigned only during badge creation.

When a badge is created, the creator supplies an array of editor addresses. The contract stores those addresses as authorized editors for that badge. After creation, there is no function to add a new editor, remove an existing editor, or replace an editor.

This means editor permissions are permanent unless the contract is upgraded. The existing editor can modify badge metadata, set badge hooks, and call other editor-gated functions, but cannot rotate the editor set itself.

**Impact:** If an editor key is lost, badge settings may become permanently unmanageable. If an editor key is compromised, the protocol cannot revoke its access without an upgrade or migration. If a community or protocol changes governance addresses, existing badges cannot be updated to follow the new governance.

This issue also compounds other risks. For example, invalid hook addresses can brick badge operations, and compromised editors cannot be removed afterward.

**Path:** contracts/Accounts/SocietyProtocolBadges.sol:92

**Recommendation:** Consider documenting clearly intended behavior.

**Found in:** c9b663b0

**Status:** Acknowledged

## I04 - Wrapper Membership Semantics Are Ambiguous

`CommunityWrapper` is meant to expose community membership through an ERC20-like `balanceOf()` function. The contract stores a list of badge IDs called `allowedBadgeIds`. The comments describe this list as a membership requirement and say that a user must hold at least one of each listed ID to be considered a member.

That wording implies an "all-of-set" rule: A user is a member only if they hold every badge ID in the list.
However, the implementation does something different. It loops through all configured badge IDs and adds up the user's balance for each one.
That is "sum" semantics: A user gets the wrapper balance from any listed badge they hold.
These two interpretations are materially different.

Example:

- The wrapper has three allowed badge IDs: A, B, and C.
- Alice holds only badge A.
- Under all-of-set semantics, Alice should not be considered a member because she does not hold B and C.
- Under current sum semantics, Alice gets a positive wrapper balance because she holds A.

This difference matters because external systems may use `CommunityWrapper.balanceOf(account) > 0` as a membership check.

**Impact:** Integrators can make incorrect access-control decisions. If an external application reads the comments or assumes "required badge IDs" means all listed badges are required, it may treat wrapper balances as stricter than they are.

**Path:** contracts/Accounts/CommunityWrapper.sol:23, contracts/Accounts/CommunityWrapper.sol:85

**Found in:** c9b663b0

**Status:** Fixed in b444df60

### I05 - ERC20 Wrapper Supply Is Inconsistent With Balances

`CommunityWrapper` presents community badge ownership through an ERC20-like interface. Its `balanceOf(account)` function dynamically calculates a user's wrapper balance by summing the user's ERC1155 balances for configured badge IDs.
However, `totalSupply()` always returns zero. This creates an ERC20 inconsistency: individual accounts can have positive balances, but the total token supply is reported as zero. For a normal ERC20 token, total supply should represent the total amount of tokens held across all accounts. In this wrapper, balances are synthetic and dynamic, so there may not be a simple stored supply value. Still, exposing the contract through the ERC20 interface while returning zero supply can confuse consumers.

Additionally the `approve()` function is not blocked with custom error.

**Impact:** ERC20 integrations may mis-handle the wrapper token.

**Path:** contracts/Accounts/CommunityWrapper.sol:85, contracts/Accounts/CommunityWrapper.sol:139

**Recommendation:** 1. Clarify the wrapper's intended ERC20 semantics. If the wrapper is only a convenience read adapter and not meant to behave like a full ERC20, document that clearly. It should be described as a synthetic, non-transferable membership view rather than a normal token. If ERC20 compatibility matters, redesign supply semantics.
2. Consider blocking the `approve()` function with custom error for consistency.

**Found in:** c9b663b0

**Status:** Fixed in b444df60

# General Risks / Assumptions

- Upgradeable Contract Trust Assumption: Core contracts use UUPS upgradeability. Users and integrators must trust the upgrade authority to preserve storage layout, maintain permission semantics, and avoid malicious or incompatible implementations.
- Badge Permission Model Complexity: Badge minting, transferring, and burning are governed by configurable rules and optional hooks rather than only standard ERC1155 approvals. Misconfigured permissions can allow authorized badge holders to operate on other users' badges.
- Hook Contract Trust and Availability: Badge hooks can override mint, transfer, burn, and balance behavior. A faulty or malicious hook can deny legitimate actions, report incorrect balances, or break integrations relying on badge ownership.
- The `acceptInvite()` function requires the message to end with exactly `Strings.toHexString(msg.sender)`, which produces a lowercase `0x`-prefixed hex string. If a user signs a message with checksummed address (mixed case), trailing punctuation or whitespace, or different formatting the signature will be rejected.
- The `CommunityWrapperFactory` assumes that wrapper owners are trusted and well-intentioned, but provides no mechanism to enforce this trust. Snapshot integrations rely on voting weight definitions being stable between proposal creation and voting, yet the wrapper owner retains the ability to modify `allowedBadgeIds` arbitrarily at any time. For directly-deployed wrappers (as opposed to Registry-managed ones), this governance attack surface is large: any deployer becomes an unfettered administrator over voting weights. The design does not account for scenarios where a wrapper owner changes weights mid-vote, either maliciously or through operator error. These assumptions are reasonable only if wrapper ownership is centralized (e.g., Registry) or if Snapshot voting always operates on block-height snapshots taken before weight changes can occur.
- The badge contract intentionally replaces the standard ERC1155 approval model with badge-rule and hook-based authorization for transfers and burns. As a result, accounts that satisfy a badge's `canTransfer` or `canBurn` rules **may be able to move or destroy tokens from other users without receiving ERC1155 approval from the token holder**. This behavior is documented in the transfer functions and appears to be part of the protocol's intended permission model, but it creates a strong custody assumption around badge configuration. Badge creators and integrators must understand that assigning a rule badge as a transferer or burner can grant all holders of that rule badge operational control over other users' balances for the target badge. The risk is especially high when rules use `PERM_EVERYONE`, broadly distributed badges, or hook-backed balances. This should be treated as an explicit protocol-level trust assumption and documented prominently for badge creators, users, and off-chain applications.

# Approach and methodology

To establish a uniform evaluation, we define the following terminology in accordance with the OWASP Risk Rating
Methodology:

| | |
|---|---|
|  | **Likelihood**<br>Indicates the probability of a specific vulnerability being discovered and exploited in real-world<br>scenarios |
|  | **Impact**<br>Measures the technical loss and business repercussions resulting from a successful attack |
|  | **Severity**<br>Reflects the comprehensive magnitude of the risk, combining both the probability of occurrence<br>(likelihood) and the extent of potential consequences (impact) |

Likelihood and impact are divided into three levels: High H, Medium M, and Low L. The severity of a risk is a blend of these two factors, leading to its classification into one of four tiers: Critical, High, Medium, or Low.

When we identify an issue, our approach may include deploying contracts on our private testnet for validation through testing. Where necessary, we might also create a Proof of Concept PoC to demonstrate potential exploitability. In particular, we perform the audit according to the following procedure:

| | |
|---|---|
|  | **Advanced DeFi Scrutiny**<br>We further review business logics, examine system operations, and place DeFi-related aspects<br>under scrutiny to uncover possible pitfalls and/or bugs |
|  | **Semantic Consistency Checks**<br>We then manually check the logic of implemented smart contracts and compare with the<br>description in the white paper. |
|  | **Security Analysis**<br>The process begins with a comprehensive examination of the system to gain a deep<br>understanding of its internal mechanisms, identifying any irregularities and potential weak spots. |